

CSW430 Mobile Application Development.

Link github: <https://github.com/thaiphanthanh sang/CSW430.git>

Question 2. Build APIs using Node Js.

The image displays three sequential screenshots of a REST client interface, likely Postman, used for testing API endpoints. Each screenshot includes a request configuration bar at the top, a body tab showing the request payload, and a response tab showing the server's reply. A Notepad window is overlaid on each response, displaying the text of the response body.

First Screenshot: Get All Users

- Request:** GET `http://localhost:3000/api/users`
- Response:** 200 OK. The JSON response is an array of two user objects.

```
[{"id": 1, "name": "First Person", "email": "first@gmail.com", "created_at": "2026-07-10T02:07:57.000Z"}, {"id": 2, "name": "Second Person", "email": "second@gmail.com", "created_at": "2026-07-10T02:08:22.000Z"}]
```

The Notepad window shows the text: `Thái Phan Thành Sang` and `2231200114`.

Second Screenshot: Create User

- Request:** POST `http://localhost:3000/api/users`
- Request Body:**

```
{ "name": "Second Person", "email": "second@gmail.com" }
```
- Response:** 201 Created. The JSON response is

```
{ "message": "User created", "userId": 2 }
```

. The Notepad window shows the same text as the first screenshot.

Third Screenshot: Get User By Id

- Request:** GET `http://localhost:3000/api/users/2`
- Response:** 200 OK. The JSON response is a single user object:

```
{ "id": 2, "name": "Second Person", "email": "second@gmail.com", "created_at": "2026-07-10T02:08:22.000Z" }
```

. The Notepad window shows the same text as the previous screenshots.

CSW 430 > Update User

PUT http://localhost:3000/api/users/2

Body

```
1 {
2   "name": "Second Person",
3   "email": "second.person@gmail.com"
4 }
```

200 OK

Body

```
1 {
2   "message": "User updated"
3 }
```

CSW 430 > Delete User

DELETE http://localhost:3000/api/users/2

Body

```
1 {
2   "message": "User deleted"
3 }
```

200 OK

Untitled - Notepad

Thái Phan Thành Sang
2231200114

Question 3. Login screen interface design.

Q2Login

App.tsx M X JS index.js JS Style.js U JS Login.js U

```
1 import React from 'react';
2 import LoginScreen from './Login';
3
4 export default function App() {
5   return <LoginScreen />;
6 }
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\Sang\CSW430\CSW430\Lab2\Question3\Q2Login> npx react-native run-android

For more on this, please refer to https://docs.gradle.org/9.3.1/userguide/command_line_tools.html.

BUILD SUCCESSFUL in 52s

82 actionable tasks: 9 executed, 73 up-to-date

Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.1/userguide/configuration_cache.html

Info Connecting to the development server...

8081

Info Starting the app on "emulator-5554"...

Starting: Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] }

main* 6 0 Java: Ready

Login

Phone

Password

Login

Question 4. Write the Personal Income Tax Calculation program as shown

Question 5. Caculator screen interface design

The screenshot shows the Android Studio IDE with the following components:

- EXPLORER (Left):** Displays the project structure for 'Q3Calculator'. The file tree includes:
 - android
 - ios
 - node_modules
 - App.tsx (selected)
 - babel.config.js
 - Gemfile
 - index.js
 - jest.config.js
 - metro.config.js
 - package-lock.json
 - package.json
 - README.md
 - styles
 - tsconfig.json
- Editor (Center):** Shows the 'App.tsx' file with the following code:


```

export default function App(): React.JSX.Element {
  return (
    <View style={styles.container}>
      <Text style={styles.welcome}>Hello World!</Text>
      <View style={styles.row}>
        <renderButton('4', () => handleNumberInput(4))>
        <renderButton('5', () => handleNumberInput(5))>
        <renderButton('6', () => handleNumberInput(6))>
        <renderButton('8', () => handleOperatorInput('*', 'operator'))>
      </View>
      <View style={styles.row}>
        <renderButton('1', () => handleNumberInput(1))>
        <renderButton('2', () => handleNumberInput(2))>
        <renderButton('3', () => handleNumberInput(3))>
        <renderButton('0', () => handleOperatorInput('.', 'operator'))>
      </View>
      <View style={styles.row}>
        <TouchableOpacity
          style={styles.button, styles.zeroButton}
          onPress={() => handleNumberInput(0)}
          activeOpacity={0.7}>
          0
        </TouchableOpacity>
        <Text style={styles.buttonText}>0</Text>
      </View>
      <View style={styles.row}>
        <renderButton('+', () => handleOperatorInput('+', 'operator'))>
        <renderButton('-', handleEqual, 'equal')>
      </View>
    </View>
  );
}

/* Clear button */
<TouchableOpacity
  style={styles.clearButton}
  onPress={handleClear}
  activeOpacity={0.7}>
  C
</TouchableOpacity>

```
- Preview (Right):** Shows a mobile emulator displaying the calculator app. The screen shows the number '10' and a numeric keypad with buttons for digits 0-9, operators (+, -, *, /), and a clear button (C).